



CHAPITRE 1 : Les Réseaux de Neurones ARTIFICIELS

DAKAR INSTITUTE OF TECHNOLOGY, MASTER IA

Module : Deep Learning

Enseignant : Mr. Moussa DIALLO

Présentation Générale de l'Intelligence Artificielle (IA)



Qu'est-ce que l'IA ?

Simulation de l'intelligence humaine par des machines, capables d'apprendre, de raisonner et de s'auto-corriger pour résoudre des problèmes complexes.

Objectif:

Créer des systèmes autonomes et adaptatifs.



Les Types d'IA

- **IA Faible (Narrow AI):** Conçue pour une tâche spécifique (ex: assistants vocaux, reconnaissance faciale).
- **IA Forte (General AI):** Capacité cognitive équivalente à l'humain, encore théorique.
- **Super IA:** Surpasse l'intelligence humaine dans tous les domaines (futur hypothétique).



Domaines d'Application Clés

- Apprentissage Automatique (Machine Learning)
- Traitement du Langage Naturel (NLP)
- Vision par Ordinateur
- Robotique & Automatisation
- Systèmes Experts & Aide à la Décision

L'IA transforme rapidement de nombreux secteurs, de la santé à la finance, en passant par les transports et l'industrie.

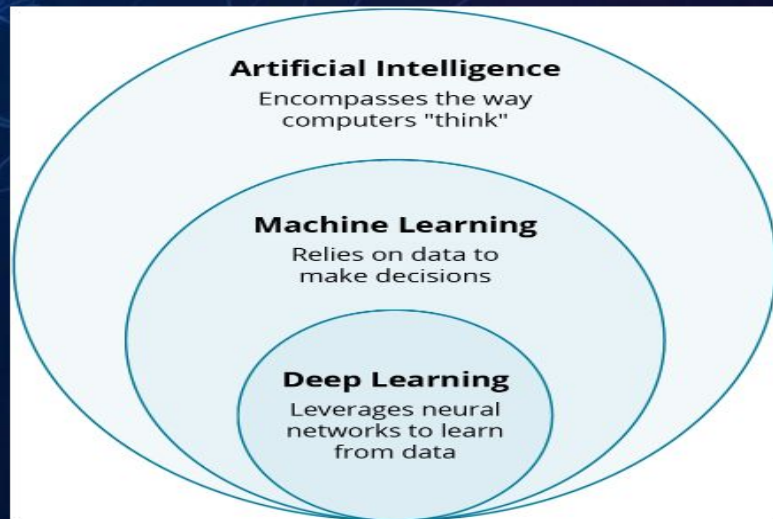
Introduction au Deep Learning

Qu'est ce que le Deep Learning ?

- ❖ Une sous-catégorie du Machine Learning
- ❖ Repose sur les Réseaux de Neurones Artificiels (RNA)

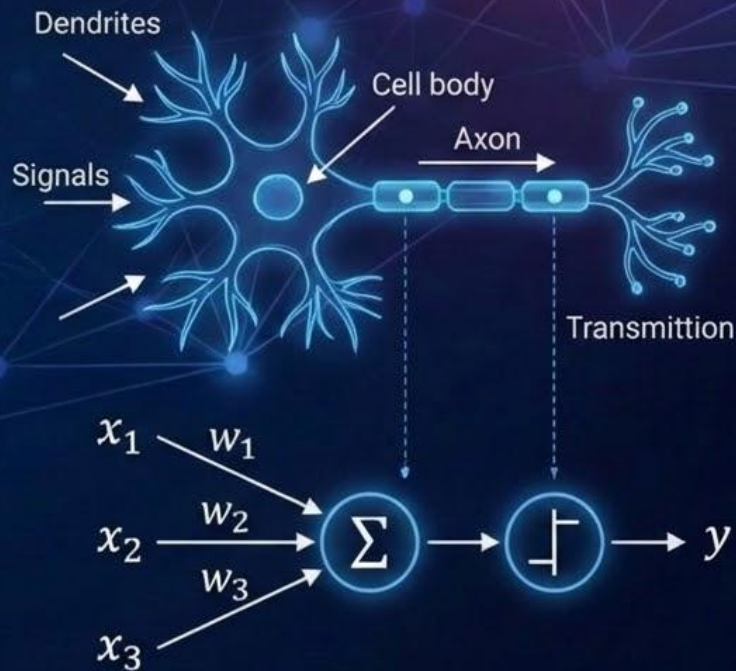
Apprentissage de tâches complexes :

- ❖ Capable d'apprendre des tâches complexes : reconnaissance d'images, conduite autonome, conversation.



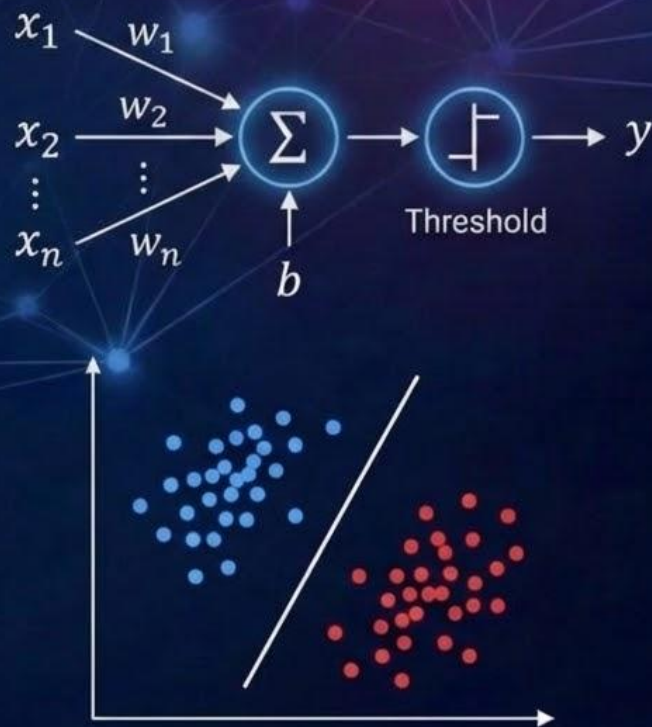
Le neurone artificiel : les origines

- ✦ Inspiration des neurones biologiques : dendrites, axone, seuil
- ✦ McCulloch & Pitts (1943) modélise un neurone comme une fonction de transfert
- ✦ Modélise des fonctions logiques simples
- ✦ Limite de cette fonction : pas d'apprentissage automatique



Le Perceptron de Rosenblatt

- Algorithme d'apprentissage inspiré de la théorie de Hebb sur la plasticité synaptique.
- Fonctionnement : un neurone artificiel s'activant lorsque la somme pondérée de ses entrées $\sum(w_i x_i + b)$ dépasse un seuil.
- Séparation linéaire des classes
- Limite : pas adapté aux problèmes non-linéaires

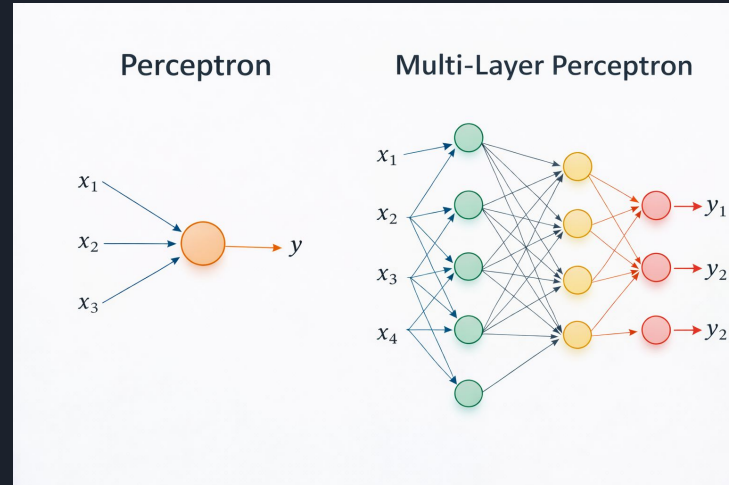


Du Perceptron au Multi Layer Perceptron

Le problème initial : Le perceptron classique était limité car il ne pouvait pas traiter les problèmes non linéaires.

La solution (Le MLP) : Le Perceptron Multicouche, développé par Geoffrey Hinton dans les années 80, permet de surmonter cet obstacle.

Architecture : C'est un réseau de neurones organisé en trois parties : une couche d'entrée, une ou plusieurs couches cachées, et une couche de sortie.





neural network playground

<https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://playground.tensorflow.org/&ved=2ahUKEwjN-8mUq52TAxVHTqQEHUnSEywQFnoECBYQAQ&usg=AOvVaw2vOMcWmUkU5HmbODfylijL>



Qu'est ce qu'un Perceptron Multicouche (MLP)

- Définition:

Un réseau neuronal artificiel qui génère un ensemble de **sorties** à partir d'un ensemble **d'entrées**.

Il est caractérisé par plusieurs **couches** de nœuds **d'entrée** connectées sous forme de graphe orienté entre les **couches d'entrée** et de **sortie**

Le MLP est une méthode d'apprentissage en profondeur (Deep Learning) qui utilise la **rétropropagation** pour apprendre.

- Structure Général:

Couche d'entrée: Reçoit les données initiales

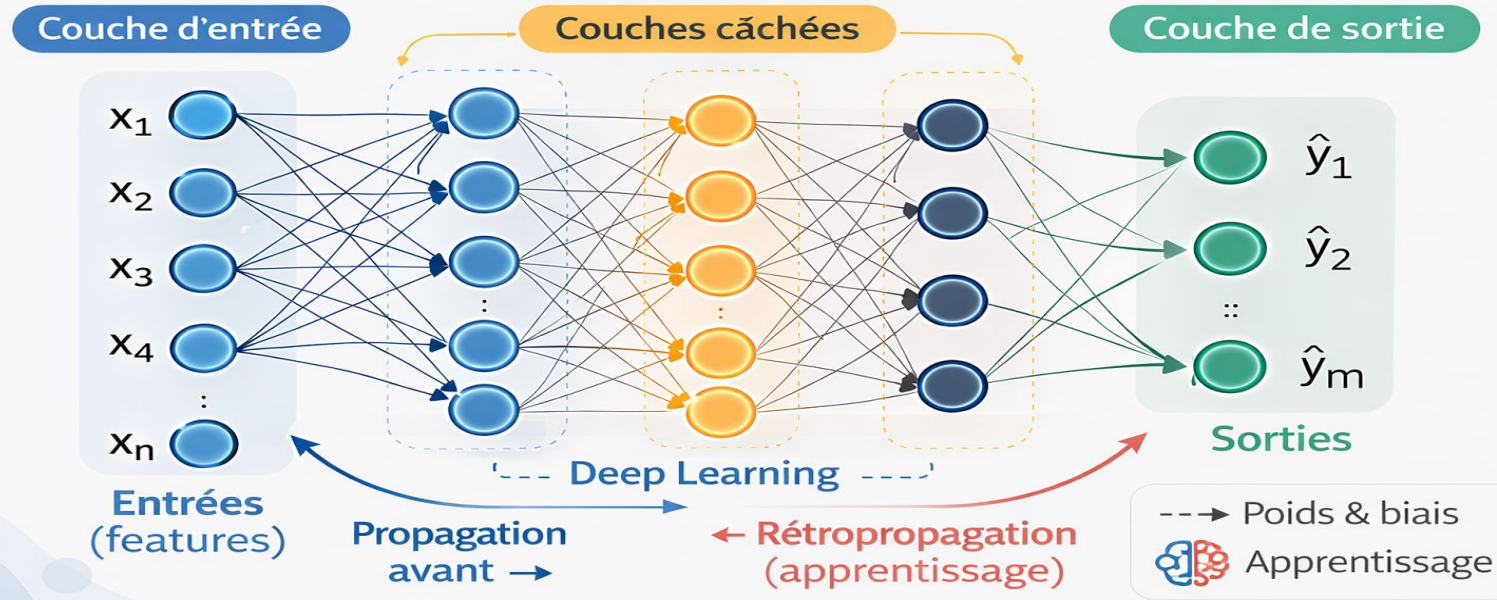
Une ou plusieurs **couches cachées**: ce qui donne le réseau sa « **profondeur** »

Couche de sortie: produit la production finale du modèle

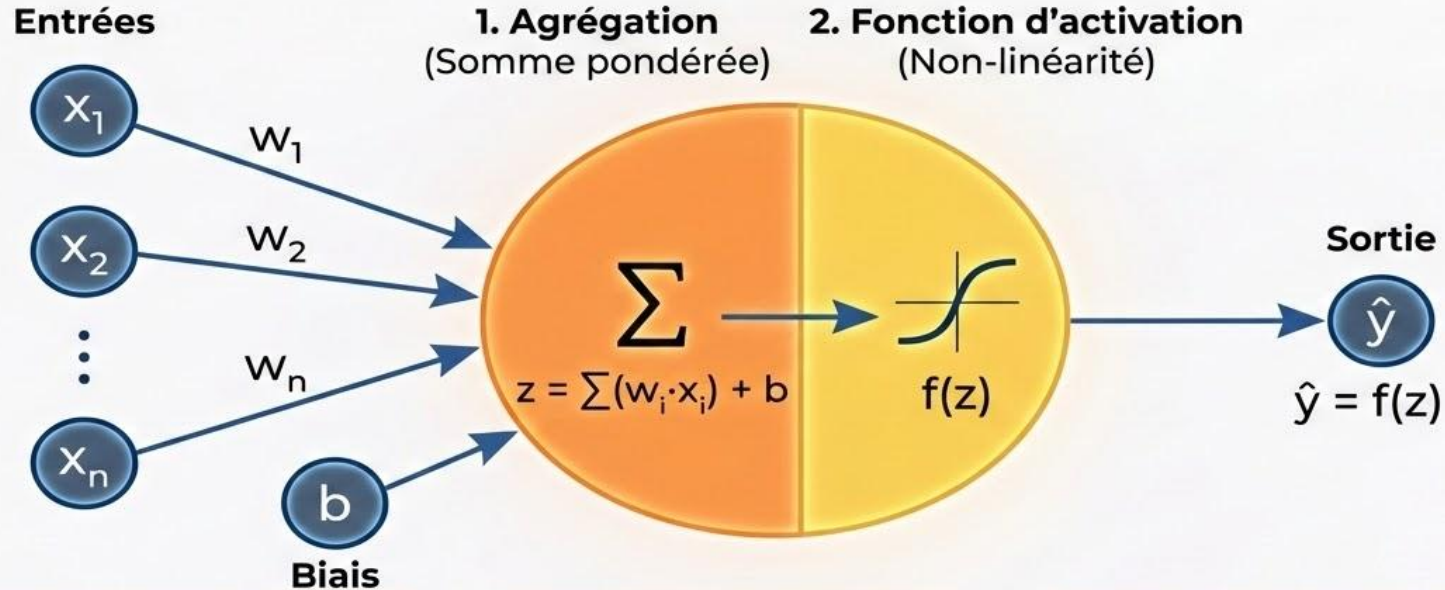
Qu'est ce qu'un Perceptron Multicouche (MLP)

MLP – Multi-Layer Perceptron

Un réseau neuronal qui génère des **sorties** à partir d'**entrées**



Le Neurone élémentaire dans le MLP



Composants

Composants: Entrées (x_i), Poids (w_i), et Biais (b).

Calcul

Calcul: La somme linéaire 'z' est calculée.

Sortie

Sortie: La fonction 'f' introduit la non-linéarité pour la sortie ' $\hat{y}$ '.

Principaux Frameworks de Deep Learning



TensorFlow

- Développé par Google
- Robuste, orienté production
- Large écosystème (TFX, TensorBoard)



PyTorch

- Développé par Meta AI
- Flexible, populaire en recherche
- Graphe de calcul dynamique



Keras

- API de haut niveau
- Simple, prototypage rapide
- S'exécute souvent sur TensorFlow



JAX

- Développé par Google
- Haute performance, flexible
- Différenciation automatique

Note : Le choix du framework dépend souvent des besoins spécifiques du projet (recherche vs production).

Initiation à Pytorch

PyTorch: a deep learning framework

- One of the most popular frameworks
- Originally developed by Meta AI, now part of Linux Foundation
- Intuitive and user-friendly
- Similarities with NumPy





Qu'est-ce qu'un tenseur ?

Définition simple :

Un tenseur est une structure de données multidimensionnelle (scalaires, vecteurs, matrices, etc.) généralisée à n dimensions.

Différence avec NumPy ndarray :

- Très similaire aux ndarray de NumPy.
- Mais optimisé pour le calcul différentiable (backpropagation).
- Compatible avec le GPU/TPU → permet des calculs massivement parallèles.

Importance en Deep Learning :

- Représentation standard des données (images, texte, audio, séries temporelles...).
- Supporte les opérations complexes (convolutions, produits matriciels, normalisation...).
- Constitue la brique de base de tous les modèles en PyTorch et TensorFlow.

PyTorch tensors

- Tensor:
 - Similar to array or matrix
 - Building block of neural networks

```
import torch
my_list = [[1, 2, 3], [4, 5, 6]]
tensor = torch.tensor(my_list)
print(tensor)
```

```
tensor([[1, 2, 3],
        [4, 5, 6]])
```

Attribut d'un tensor

- shape → taille
- dtype → type des données
- device → CPU ou GPU

Exemple :

```
tensor = torch.rand(3,4)  
print(tensor.shape, tensor.dtype, tensor.device)
```

```
torch.Size([3, 4]) torch.float32 cpu
```

Utilisation du GPU (CUDA) avec PyTorch

- Accélération significative des calculs matriciels pour le Deep Learning.
- Nécessite une carte graphique NVIDIA compatible CUDA.

```
import torch

# Vérifier la disponibilité de CUDA
if torch.cuda.is_available():
    print('CUDA est disponible!')
    device = torch.device('cuda')
else:
    print('CUDA n\'est pas disponible. Utilisation du CPU.')
    device = torch.device('cpu')
```

- Déplacer les tenseurs et les modèles vers le GPU :

```
# Déplacer un tenseur
x = torch.randn(3, 3)
x_gpu = x.to(device)

# Déplacer un modèle (exemple)
# model = MonModele()
# model = model.to(device)
```


Tensor attributes

- Tensor shape

```
my_list = [[1, 2, 3], [4, 5, 6]]  
tensor = torch.tensor(my_list)  
print(tensor.shape)
```

```
torch.Size([2, 3])
```

- Tensor data type

```
print(tensor.dtype)
```

```
torch.int64
```

Getting started with tensor operations

Compatible shapes

```
a = torch.tensor([[1, 1],  
                  [2, 2]])  
  
b = torch.tensor([[2, 2],  
                  [3, 3]])
```

- Addition / subtraction

```
print(a + b)
```

```
tensor([[3, 3],  
        [5, 5]])
```

Incompatible shapes

```
a = torch.tensor([[1, 1],  
                  [2, 2]])  
  
c = torch.tensor([[2, 2, 4],  
                  [3, 3, 5]])
```

- Addition / subtraction

```
print(a + c)
```

```
RuntimeError: The size of tensor a  
(2) must match the size of tensor b (3)  
at non-singleton dimension 1
```



Element-wise multiplication

```
a = torch.tensor([[1, 1],  
                  [2, 2]])  
b = torch.tensor([[2, 2],  
                  [3, 3]])  
  
print(a * b)
```

```
tensor([[2, 2],  
        [6, 6]])
```

Matrix multiplication

```
a = torch.tensor([[1, 1],  
                  [2, 2]])  
b = torch.tensor([[2, 2],  
                  [3, 3]])  
  
print(a @ b)
```

```
tensor([[5, 5],  
        [10, 10]])
```


Matrix multiplication

```
a = torch.tensor([[1, 1],  
                  [2, 2]])  
b = torch.tensor([[2, 2],  
                  [3, 3]])  
  
print(a @ b)
```

```
tensor([[5,  5],  
        [10, 10]])
```

- $1 \cdot 2 + 1 \cdot 3 = 5$
- Perform addition and multiplication to process data and learn patterns

Backpropagation avec Pytorch

```
import torch

x = torch.tensor(-3., requires_grad=True)
y = torch.tensor(5., requires_grad=True)
z = torch.tensor(-2., requires_grad=True)

q = x + y
f = q * z

f.backward()

print("Gradient of z is: " + str(z.grad))
print("Gradient of y is: " + str(y.grad))
print("Gradient of x is: " + str(x.grad))
```

```
Gradient of z is: tensor(2.)
Gradient of y is: tensor(-2.)
Gradient of x is: tensor(-2.)
```

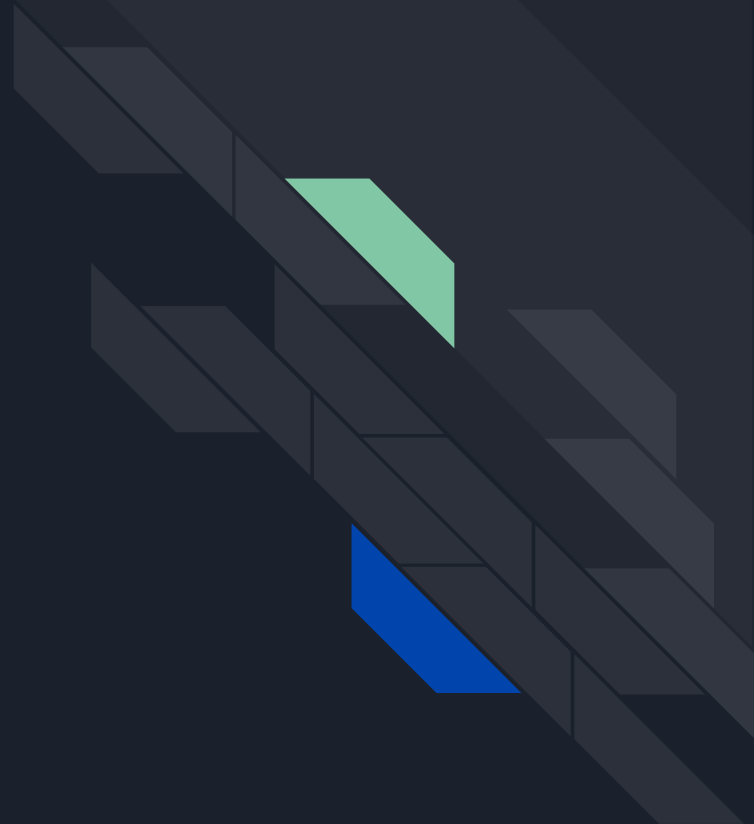

$$f = z^*(x+y)$$

$$df/z = x+y = -3+5 = 2$$

$$df/dz = 2$$

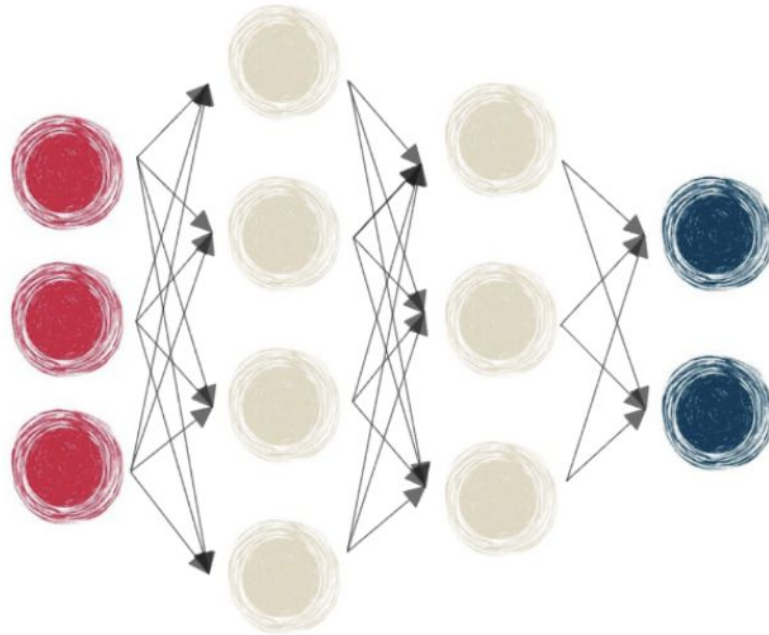
$$df/dy = z = -2$$

NEURAL NETWORK AND LAYER

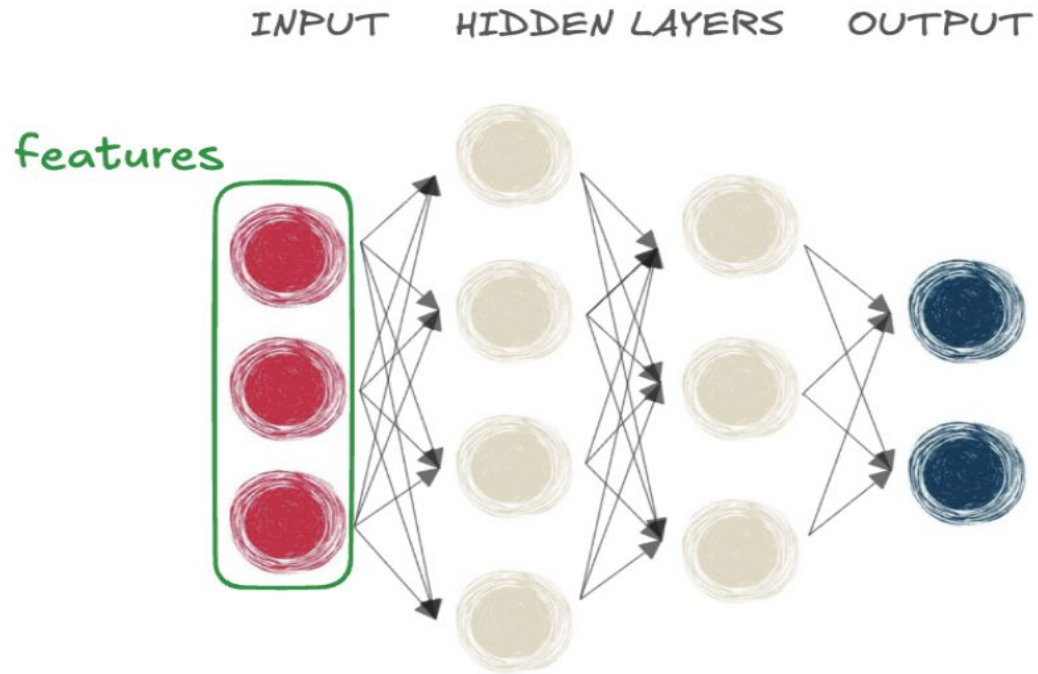


Neural network layers

INPUT HIDDEN LAYERS OUTPUT

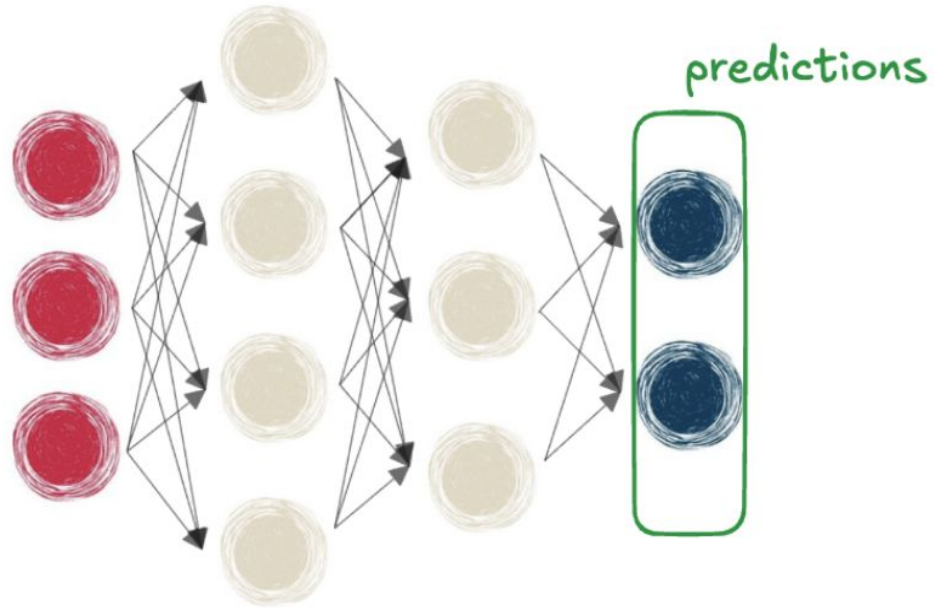


Neural network layers



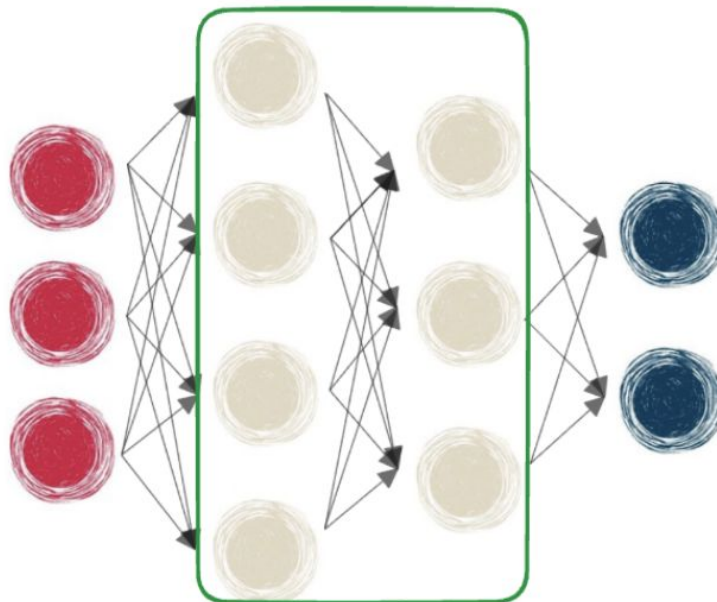
Neural network layers

INPUT HIDDEN LAYERS OUTPUT

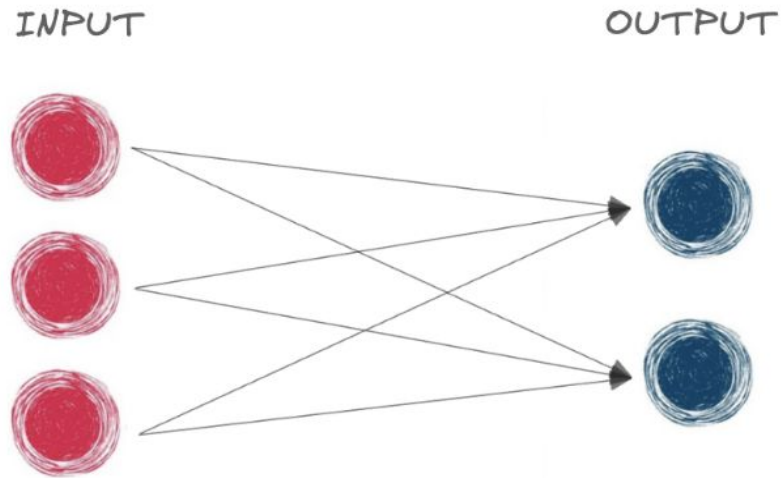


Neural network layers

INPUT HIDDEN LAYERS OUTPUT



Our first neural network



- Fully connected network
- Equivalent to a linear model

Designing a neural network

INPUT

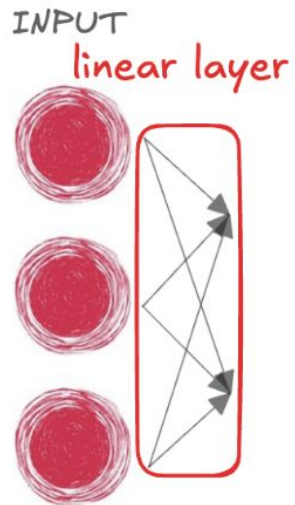


```
# Importing as nn to avoid writing torch.nn
import torch.nn as nn

# Create input_tensor with three features
input_tensor = torch.tensor(
    [[0.3471, 0.4547, -0.2356]])
```

- Input neurons = features
- Output neurons = classes

Designing a neural network



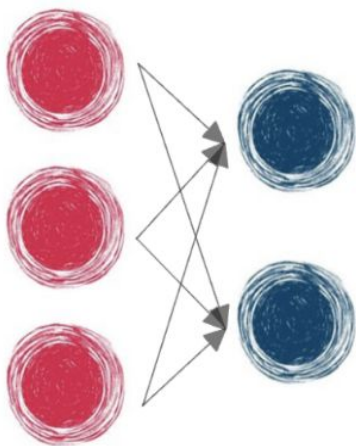
```
# Importing as nn to avoid writing torch.nn
import torch.nn as nn

# Create input_tensor with three features
input_tensor = torch.tensor(
    [[0.3471, 0.4547, -0.2356]])

# Define our linear layer
linear_layer = nn.Linear(
    in_features=3,
    out_features=2
)
```


Designing a neural network

INPUT OUTPUT



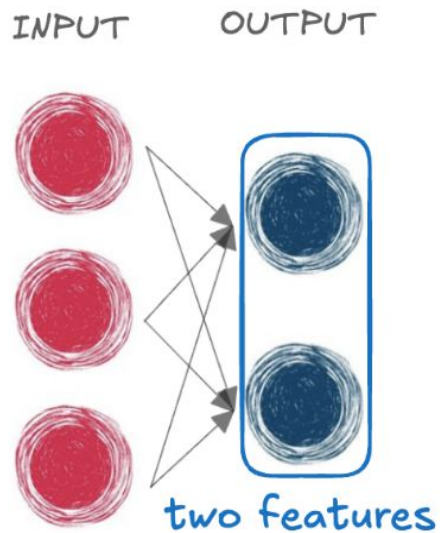
```
# Importing as nn to avoid writing torch.nn
import torch.nn as nn
```

```
# Create input_tensor with three features
input_tensor = torch.tensor(
    [[0.3471, 0.4547, -0.2356]])
```

```
# Define our linear layer
linear_layer = nn.Linear(
    in_features=3,
    out_features=2
)
```

```
# Pass input through linear layer
output = linear_layer(input_tensor)
print(output)
```

Designing a neural network



```
# Pass input through linear layer  
output = linear_layer(input_tensor)  
print(output)
```

```
tensor([[ -0.2415,  -0.1604]],  
        grad_fn=<AddmmBackward0>)
```

Weights and biases

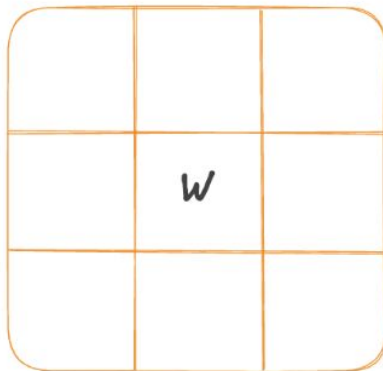
```
output = linear_layer(input_tensor)
```

INPUT



@

WEIGHTS



+

BIASES



=

OUTPUT



Weights and biases

- `.weight`

```
print(linear_layer.weight)
```

```
Parameter containing:
tensor([[ -0.4799,  0.4996,  0.1123],
        [ -0.0365, -0.1855,  0.0432]],
        requires_grad=True)
```

- Reflects the importance of different features

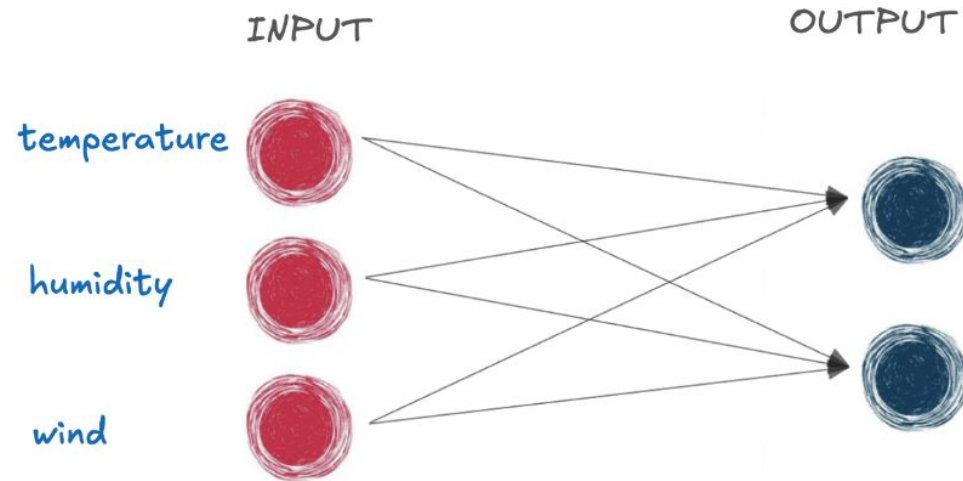
- `.bias`

```
print(linear_layer.bias)
```

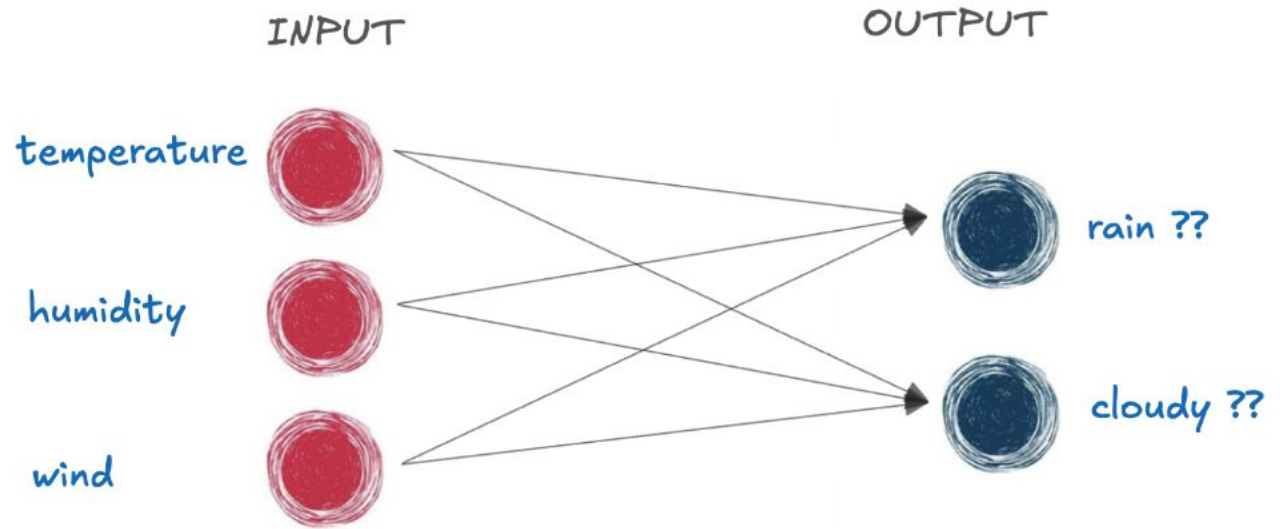
```
Parameter containing:
tensor([0.0310, 0.1537], requires_grad=True)
```

- Provides the neuron with a baseline output

A fully connected network in action

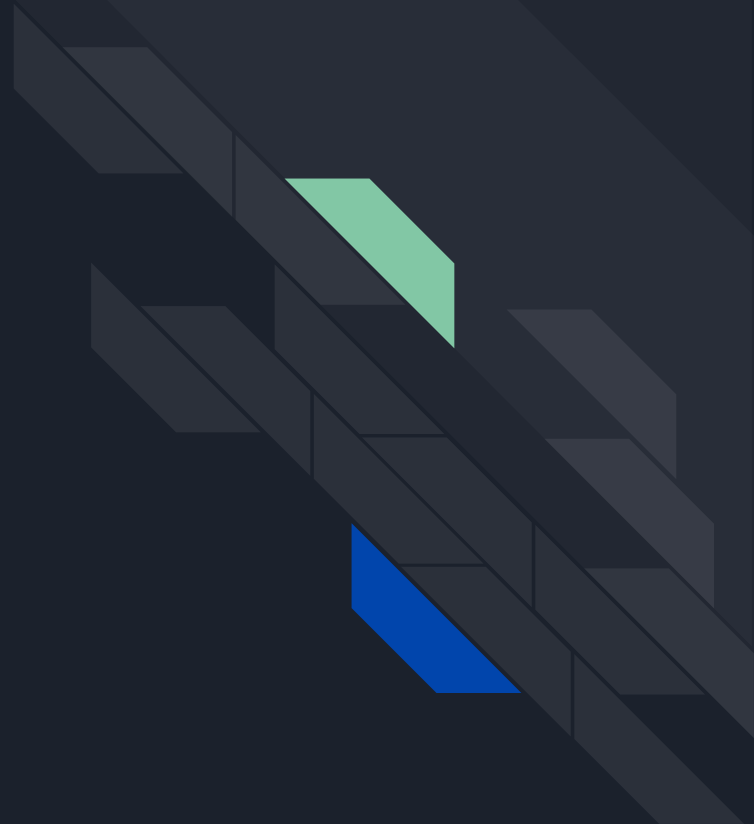


A fully connected network in action



- Humidity feature will have a more significant **weight**
- **Bias** is to account for baseline information

Couches cachée et
paramètres



Stacking layers with nn.Sequential()

```
# Create network with three linear layers
```

```
model = nn.Sequential(  
    nn.Linear(n_features, 8),  
    nn.Linear(8, 4),  
    nn.Linear(4, n_classes)  
)
```

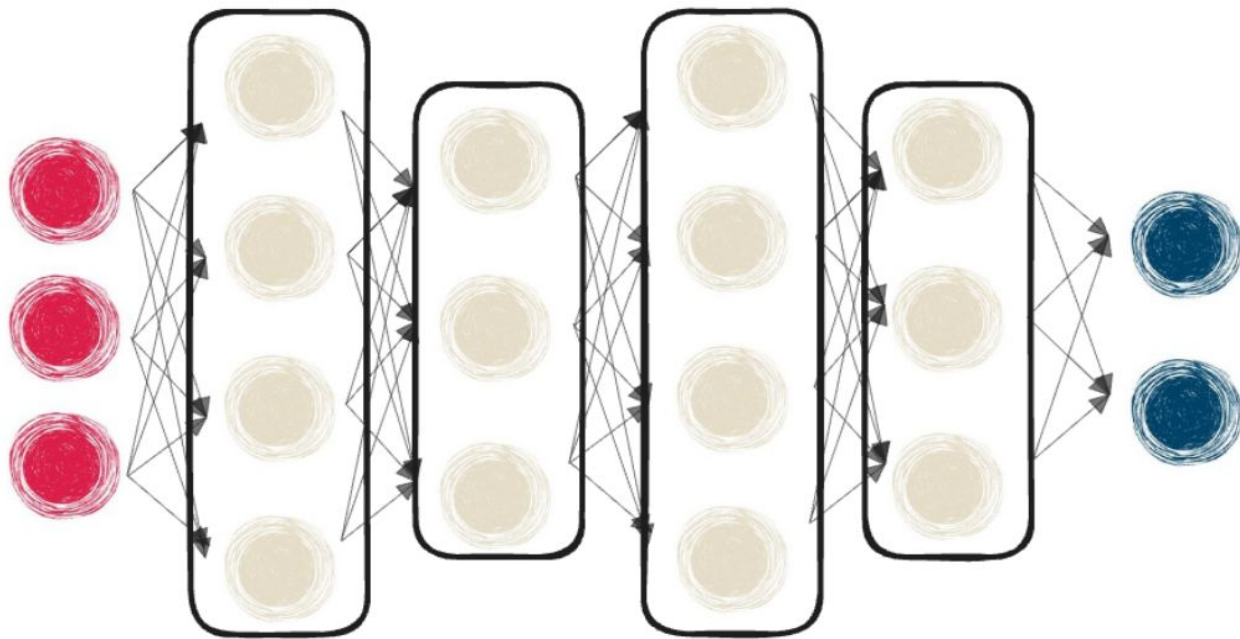
- Input is passed through the linear layers
- Layers within `nn.Sequential()` are hidden layers

Stacking layers with nn.Sequential()

```
# Create network with three linear layers
model = nn.Sequential(
    nn.Linear(n_features, 8), # n_features represents number of input features
    nn.Linear(8, 4),
    nn.Linear(4, n_classes) # n_classes represents the number of output classes
)
```

- Input is passed through the linear layers
- Layers within `nn.Sequential()` are hidden layers
- `n_features` and `n_classes` are defined by the dataset

Adding layers



Adding layers

```
# Create network with three linear layers
model = nn.Sequential(
    nn.Linear(10, 18),
    nn.Linear(18, 20),
    nn.Linear(20, 5)
)
```



Adding layers

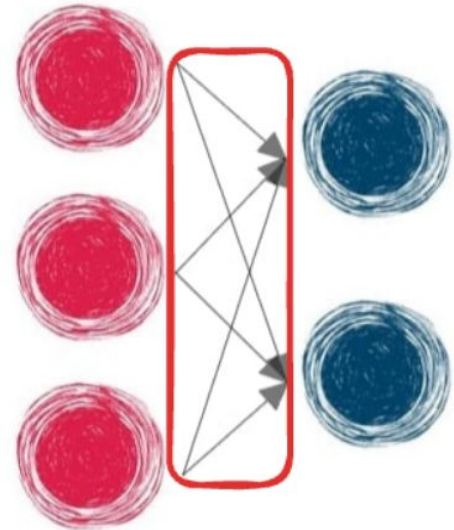
```
# Create network with three linear layers
model = nn.Sequential(
    nn.Linear(10, 18), # Takes 10 features and outputs 18
    nn.Linear(18, 20),
    nn.Linear(20, 5)
)
```

- Input 10 → output 18 → output 20 → Output 5

Layers are made of neurons

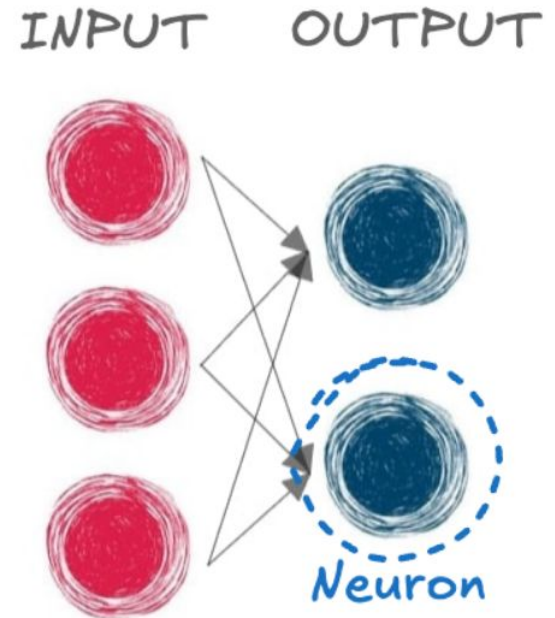
- **Fully connected** when each neuron links to all neurons in the previous layer

INPUT OUTPUT



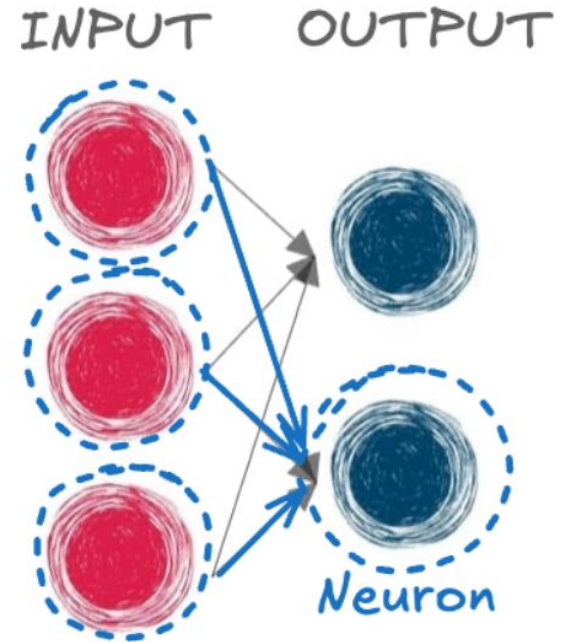
Layers are made of neurons

- **Fully connected** when each neuron links to all neurons in the previous layer
- A neuron in a linear layer:



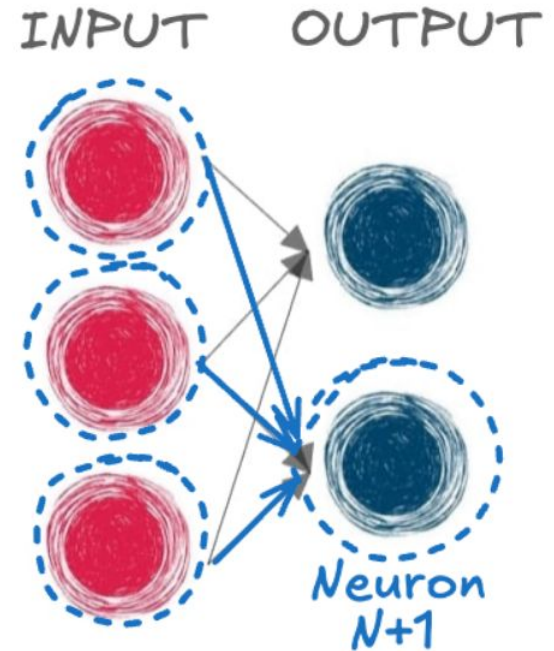
Layers are made of neurons

- **Fully connected** when each neuron links to all neurons in the previous layer
- A neuron in a linear layer:
 - Performs a linear operation using **all neurons** from the previous layer



Layers are made of neurons

- **Fully connected** when each neuron links to all neurons in the previous layer
- A neuron in a linear layer:
 - Performs a linear operation using **all neurons** from the previous layer
 - Has **$N+1$** parameters: N from **inputs** and 1 for the **bias**



Parameters and model capacity

- More hidden layers = more parameters = higher **model capacity**

Given the following model:

```
model = nn.Sequential(nn.Linear(8, 4),  
                      nn.Linear(4, 2))
```

Parameters and model capacity

- More hidden layers = more parameters = higher **model capacity**

Given the following model:

```
model = nn.Sequential(nn.Linear(8, 4),  
                      nn.Linear(4, 2))
```

Manual parameter calculation:

- First layer has 4 neurons, each neuron has 8+1 parameters. 4 times 9 = 36 parameters

Parameters and model capacity

- More hidden layers = more parameters = higher **model capacity**

Given the following model:

```
model = nn.Sequential(nn.Linear(8+1, 4),  
                      nn.Linear(4, 2))
```

Manual parameter calculation:

- First layer has 4 neurons, each neuron has 8+1 parameters. 4 times 4 = 36 parameters

Parameters and model capacity

- More hidden layers = more parameters = higher **model capacity**

Given the following model:

```
model = nn.Sequential(nn.Linear(8, 4),  
                      nn.Linear(4, 2))
```

Manual parameter calculation:

- First layer has 4 neurons, each neuron has $8+1$ parameters. $9 \times 4 = 36$ parameters
- Second layer has 2 neurons, each neuron has $4+1$ parameters. $5 \times 2 = 10$ parameters

Parameters and model capacity

- More hidden layers = more parameters = higher **model capacity**

Given the following model:

```
model = nn.Sequential(nn.Linear(8, 4),  
                      nn.Linear(4, 2))
```

Manual parameter calculation:

- First layer has 4 neurons, each neuron has $8+1$ parameters. $9 \times 4 = 36$ parameters
- Second layer has 2 neurons, each neuron has $4+1$ parameters. $5 \times 2 = 10$ parameters
- $36 + 10 = 46$ learnable parameters

Parameters and model capacity

- More hidden layers = more parameters = higher **model capacity**

Given the following model:

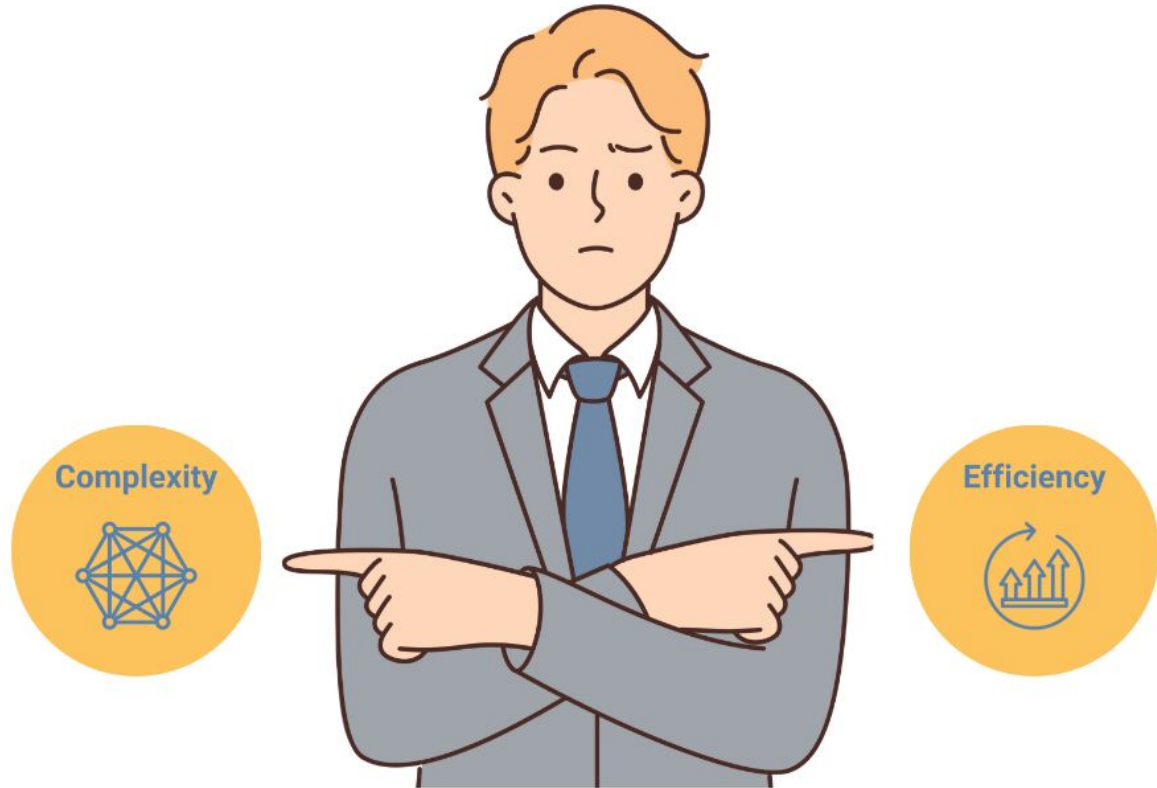
```
model = nn.Sequential(nn.Linear(8, 4),  
                      nn.Linear(4, 2))
```

Using PyTorch:

- `.numel()` : returns the number of elements in the tensor

```
total = 0  
for parameter in model.parameters():  
    total += parameter.numel()  
print(total)
```

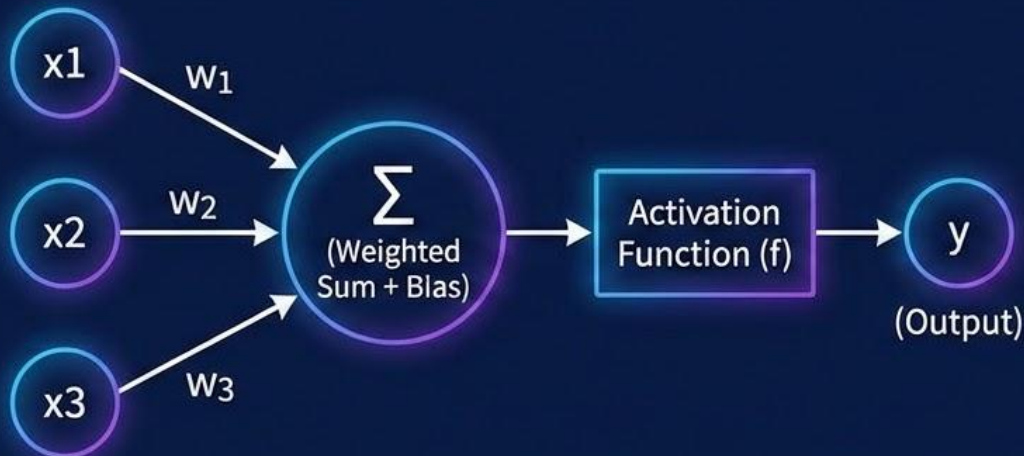
Balancing complexity and efficiency



Deep Learning: The Perceptron

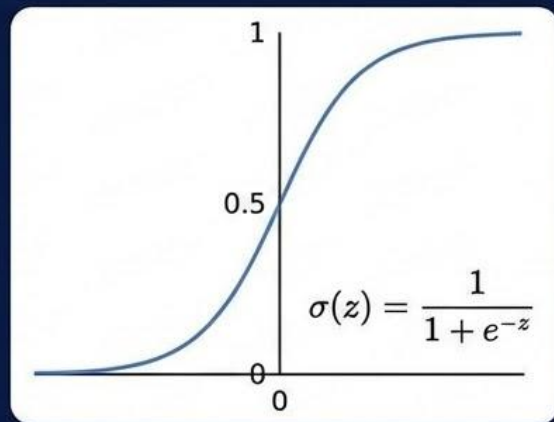
Forward Propagation & Activation Functions

The Perceptron & Forward Propagation



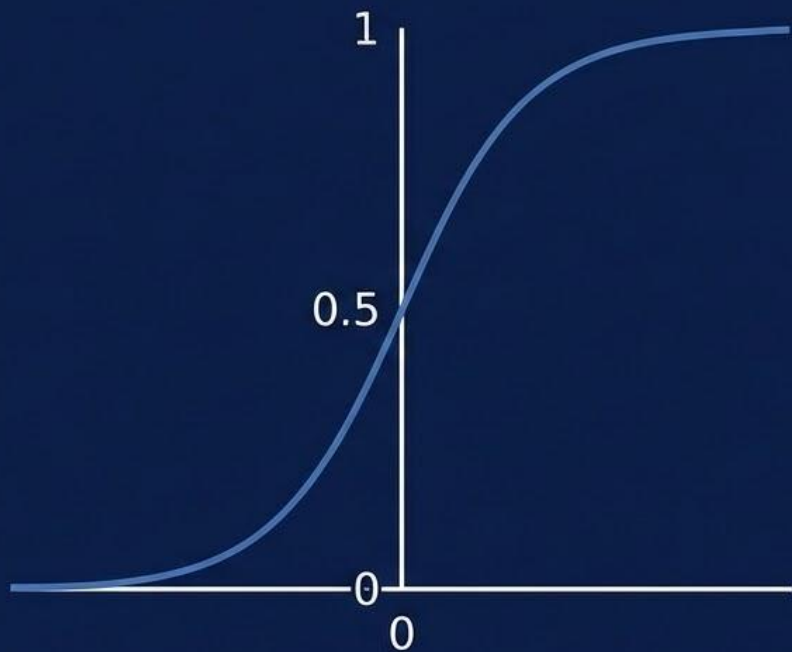
Inputs are multiplied by weights, summed with a bias, and passed through an activation function.

Activation (Intro Sigmoid)



- Introduces non-linearity
- Squashes output between 0 and 1
- Used in binary classification

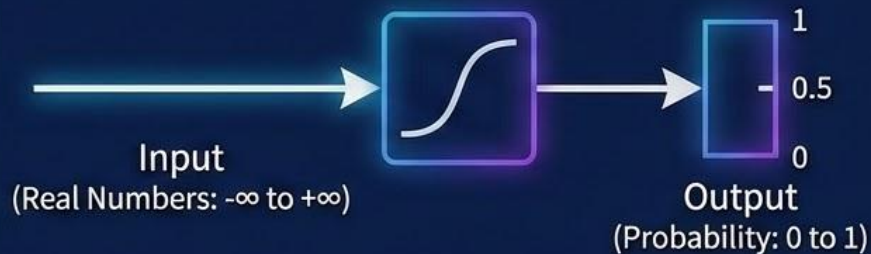
The Sigmoid Activation Function



Visual Illustration: S-shaped curve, squashes inputs to (0, 1) range.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

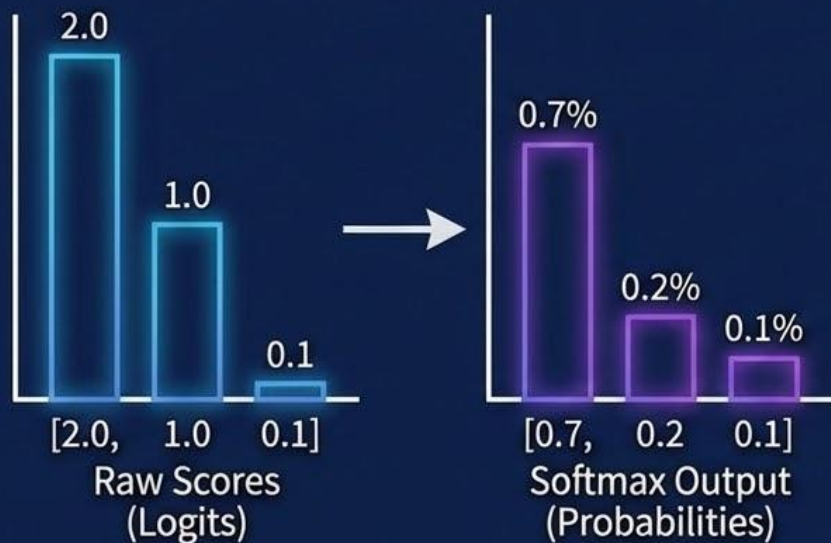
Formula: Calculates probability-like output.



Graphic: Maps any real value to a value between 0 and 1, useful for binary classification.

The Softmax Activation Function

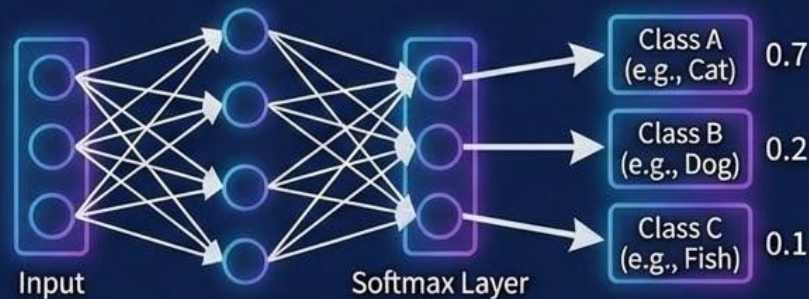
Visual Illustration: Multi-Class Probabilities



Visual Illustration: Multi-Class Probabilities
Converts raw scores into a probability distribution.
Sum of outputs is always 1.

$$\sigma(z)_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

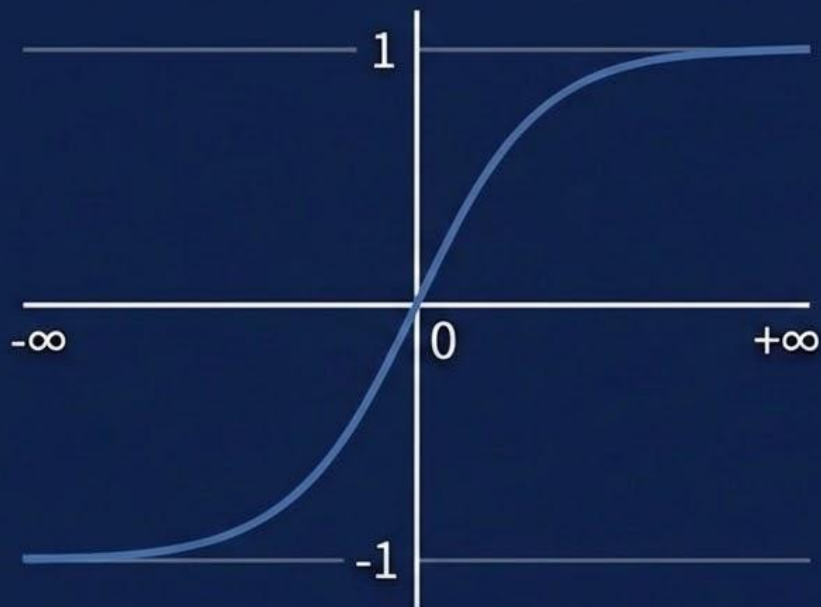
Calculates the probability of each class 'i' relative to all classes 'j'. Used for multi-class classification.



Graphic: Neural Network Output Layer
Maps network outputs to distinct class probabilities.

The Tanh Activation Function

Visual Illustration: Zero-Centered S-Curve

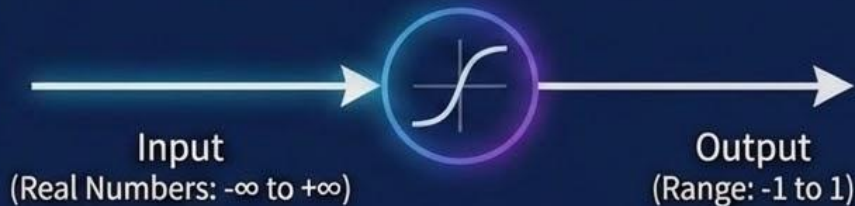


S-shaped curve, maps inputs to $(-1, 1)$ range.
Output is zero-centered.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Also related to sigmoid: $\tanh(x) = 2\sigma(2x) - 1$.
Calculates output between -1 and 1.

Graphic: Neural Network Node



Maps any real value to a value between -1 and 1.
Often preferred for hidden layers.

Fonction de Perte (Loss Function)

Le Cœur de l'Apprentissage Automatique :

- Développer un modèle en minimisant les erreurs entre les prédictions et les données réelles.

Fonction de coût : Quantifie les erreurs du modèle.

Erreur Quadratique Moyenne (MSE)

- Utilisée en Régression

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

- Mesure la distance entre valeurs réelles et prédites
- Lisse et différentiable

Entropie Croisée (Cross-Entropy)

- Utilisée en Classification

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

- Pénalise fortement les mauvaises prédictions confiantes
- Alignée avec une sortie Sigmoid (binaire) ou Softmax (multi-classes)

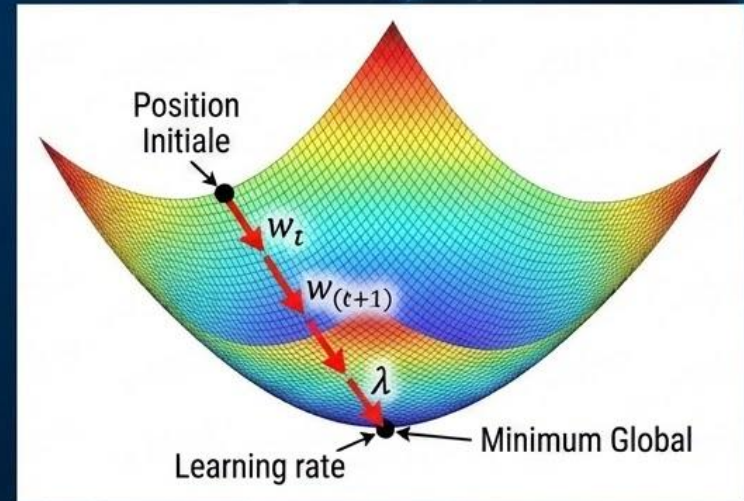
Fonction de coût et Descente du gradient

Algorithme de Descente de Gradient

- Ajuste itérativement les paramètres (poids w , et biais b) pour minimiser la fonction de coût
- Utilise les gradients (dérivées partielles) de la fonction de coût pour indiquer la direction de l'ajustement

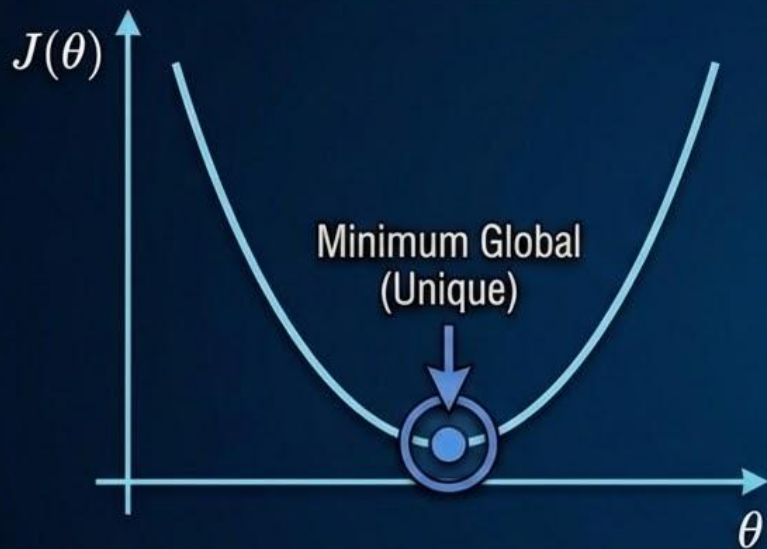
$$w_{(t+1)} = w_t - \lambda * \textit{gradient}$$

λ définit la taille des pas lors de l'ajustement (Learning rate).



Fonctions Convexes vs Non Convexes : Minimum Local et Global

Fonction Convexe



Une seule vallée. L'optimisation (ex: descente de gradient) trouve toujours le minimum global.

Fonction Non Convexe



Plusieurs vallées. L'optimisation peut rester coincée dans un minimum local, manquant le minimum global.

Implications pour l'Entraînement des Réseaux de Neurones (Deep Learning)

Deep Learning : Comprendre le fonctionnement de la rétropropagation

6. Se concentrer sur l'optimisation de la perte (Descente de gradient)



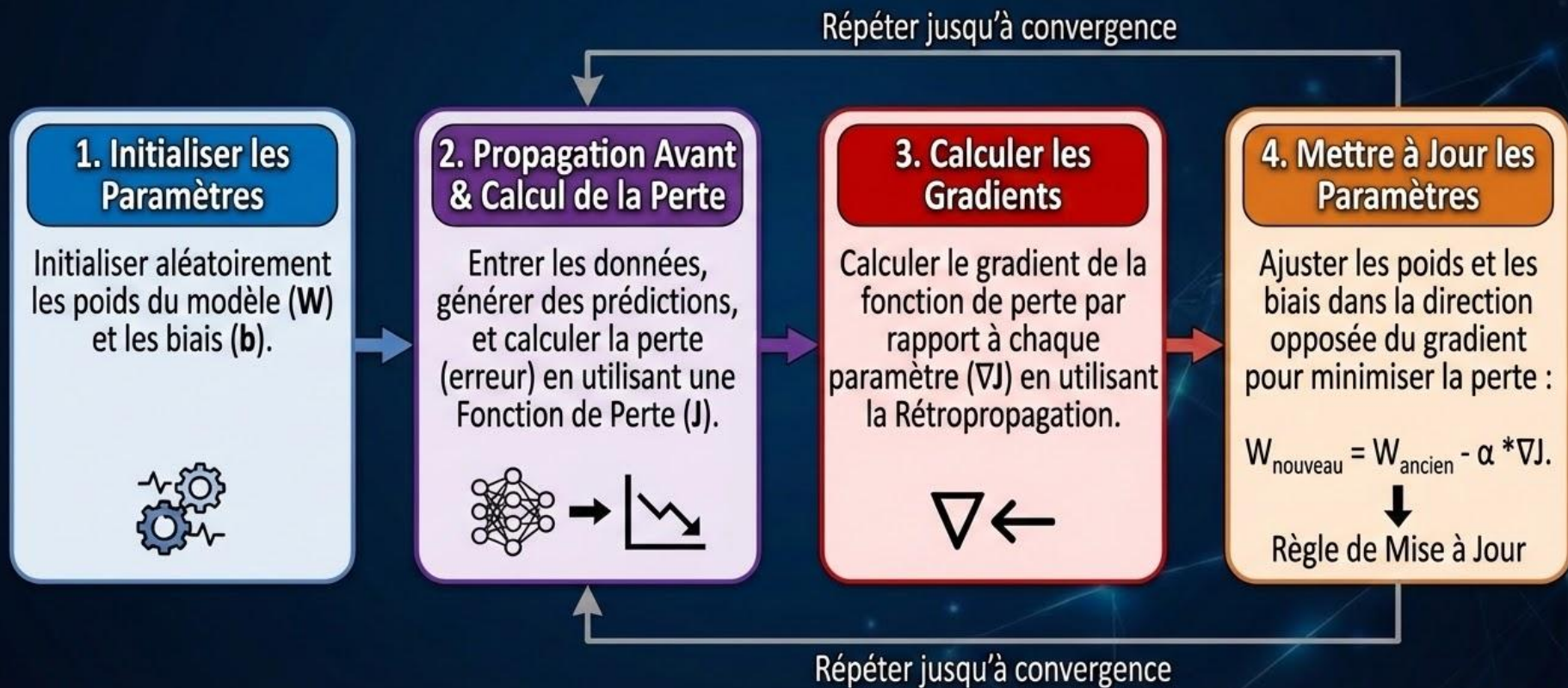
Formule de calcul du gradient:

$$\nabla J(\theta) = \left[\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots, \frac{\partial J}{\partial \theta_n} \right]^T$$

- **Descente de gradient** : Un algorithme d'optimisation itératif utilisé pour minimiser la fonction de perte.
- **Objectif** : Trouver la valeur minimale de la fonction en se déplaçant dans la direction du gradient négatif.
- **Mécanisme** : Calcule le gradient (direction de la montée la plus raide) et met à jour les poids dans la direction opposée.

Apprentissage Profond : Comprendre l'Algorithme de Descente de Gradient

7. Focus sur l'Explication Étape par Étape de l'Algorithme de Descente de Gradient

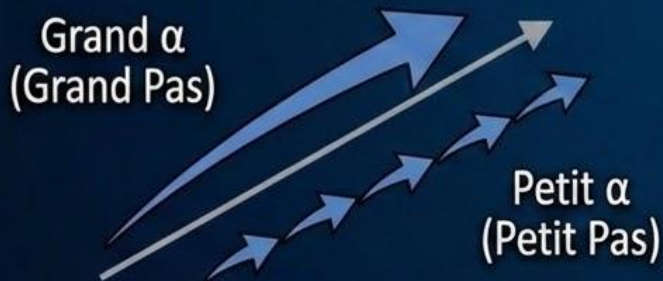


La Formule de Mise à Jour des Poids : Simple et Intuitive

$$W_{\text{nouveau}} = W_{\text{ancien}} - \alpha * \nabla J$$

Le Rôle de α (Taux d'Apprentissage)

C'est la taille du pas.



Détermine la vitesse à laquelle on ajuste les poids.

Pourquoi le Signe "Moins" (-) ?

Pour descendre vers le minimum.



Le gradient pointe vers la montée. Le signe "-" nous force à aller dans la direction opposée, pour minimiser l'erreur.

PyTorch : Fonctionnement Étape par Étape de Dataset et DataLoader

1. Données Brutes (Stockage)



Source initiale des données non structurées sur le disque.

2. La Classe Dataset (L'Organisateur)

```
class CustomDataset  
(torch.utils.data.Dataset):  
    __len__: Taille totale  
    __getitem__(idx): Charge UN  
échantillon et son étiquette
```



Définit **COMMENT** accéder à un élément unique indexé.

3. Le DataLoader (Le Livreur & Lotisseur)

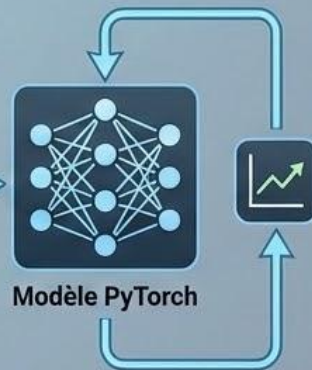
`torch.utils.data.DataLoader`



BATCH
(Lot de Tenseurs)

Iterateur qui charge, mélange et regroupe les échantillons en lots efficaces.

4. Boucle d'Entraînement (Le Modèle)



Le modèle reçoit un lot complet (ex: 64 images) pour une étape d'optimisation.

nn.Module : La Brique de Base pour la Modélisation PyTorch

nn.Module est la classe de base pour tous les modules de réseaux de neurones dans PyTorch. Elle encapsule les paramètres, les couches et définit le flux de données (forward pass).

Méthodes Clés & Attributs

- **__init__()** : Initialise les couches et sous-modules.
- **forward()** : Définit le calcul effectué à chaque appel.
- **parameters()** : Itérateur sur les paramètres du module (poids, biais).
- **to(device)** : Déplace le module et ses paramètres sur un appareil (CPU/GPU).

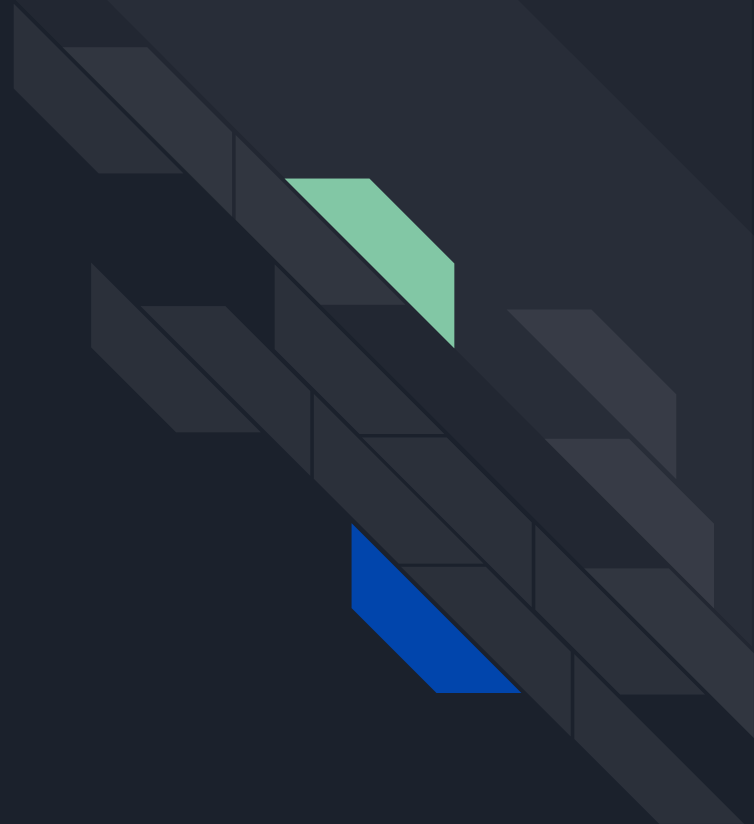
Exemple de Structure

```
import torch.nn as nn

class MonModele(nn.Module):
    def __init__(self):
        super().__init__()
        self.couche1 = nn.Linear(10, 5)
        self.activation = nn.ReLU()

    def forward(self, x):
        return self.activation(self.couche1(x))
```

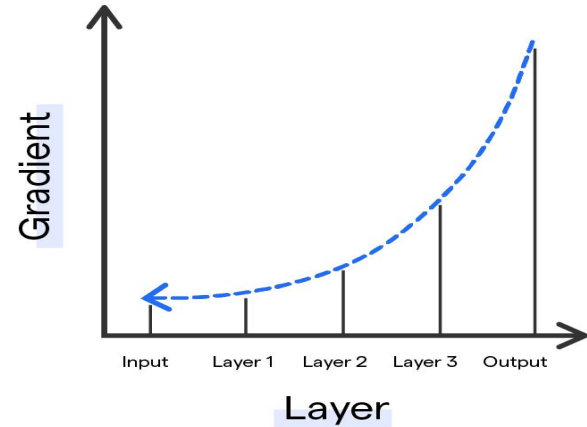

vanishing, exploding
gradient descent



Gradient qui disparaît (vanishing gradient descent)

- Les gradients deviennent **de plus en plus petits** lors de la rétropropagation
- Les premières couches reçoivent des **mise à jour très faibles**
- **Le modèle apprend peu ou pas du tout**
- **Le réseau profond devient difficile à entraîner**

Vanishing Gradient Problem





Gradient qui explose (exploding gradient descent)

- Les gradients deviennent **de plus en plus grand** lors de la rétropropagation
- Mise à jour des paramètres trop importantes
- **L'entraînement qui diverge**

SOLUTION aux gradients instables

SOLUTION



BIEN INITIALISER LES POIDS



UTILISER LES FONCTIONS
D'ACTIVATION ADÉQUATES



NORMALISER LES COUCHES
AVEC BATCH
NORMALISATION



SOLUTION aux gradients instables

1. BIEN INITIALISER LES POIDS

une bonne initialisation garantit

variance des entrées = variance des sorties

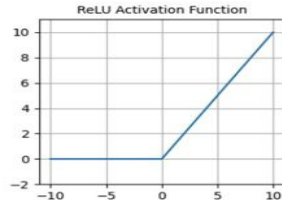
variance des gradients identiques avant et après chaque couche

La méthode dépend de la fonction d'activation:

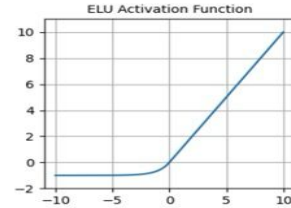
Pour “Relu” et similaire utiliser la méthode He/Kaiming

SOLUTION aux gradients instables

FONCTION D'ACTIVATION



- Often used as the default activation
- `nn.functional.relu()`
- Zero for negative inputs - dying neurons



- `nn.functional.elu()`
- Non-zero gradients for negative values - helps against dying neurons
- Average output around zero - helps against vanishing gradients

SOLUTION aux gradients instables

Batch Normalisation

- **Après une couche :**
- Normaliser les sorties de la couche :
 - En soustrayant la moyenne
 - En divisant par l'écart-type
- Appliquer un **redimensionnement et un décalage** grâce à des paramètres appris
- ➡ Le modèle apprend ainsi une distribution optimale des entrées pour chaque couche :
- Diminution plus rapide de la perte
- Réduction des problèmes de gradients instables